

Final Project Part 1

Introduction

This project assumes the role of a data analyst working for an e-commerce company that sells products across various categories. The goal is to integrate, clean, and analyze customer feedback collected from three distinct sources:

- Customer Surveys – structured CSV data (customer_survey.csv)
- Web Feedback Forms – semi-structured JSON data (web_feedback.json)
- External Review Platforms – XML data (external_reviews.xml)

The analytical objectives are: (1) identify top-performing products based on ratings, (2) highlight recurring issues or complaints, and (3) perform basic sentiment analysis through rating trends and keyword frequency.

2. Database Design

2.1 Entities and Attributes

The relational database is composed of three normalized entities:

Entity	Primary Key	Attributes
customers	customer_id (INT)	name, email (UNIQUE), region
products	product_id (INT)	name, category, price
reviews	review_id (INT)	customer_id (FK), product_id (FK), rating, comments, review_date, source

All schemas are in Third Normal Form (3NF): no transitive dependencies, each non-key attribute depends solely on the primary key of its table.

2.2 Relationships and Foreign Keys

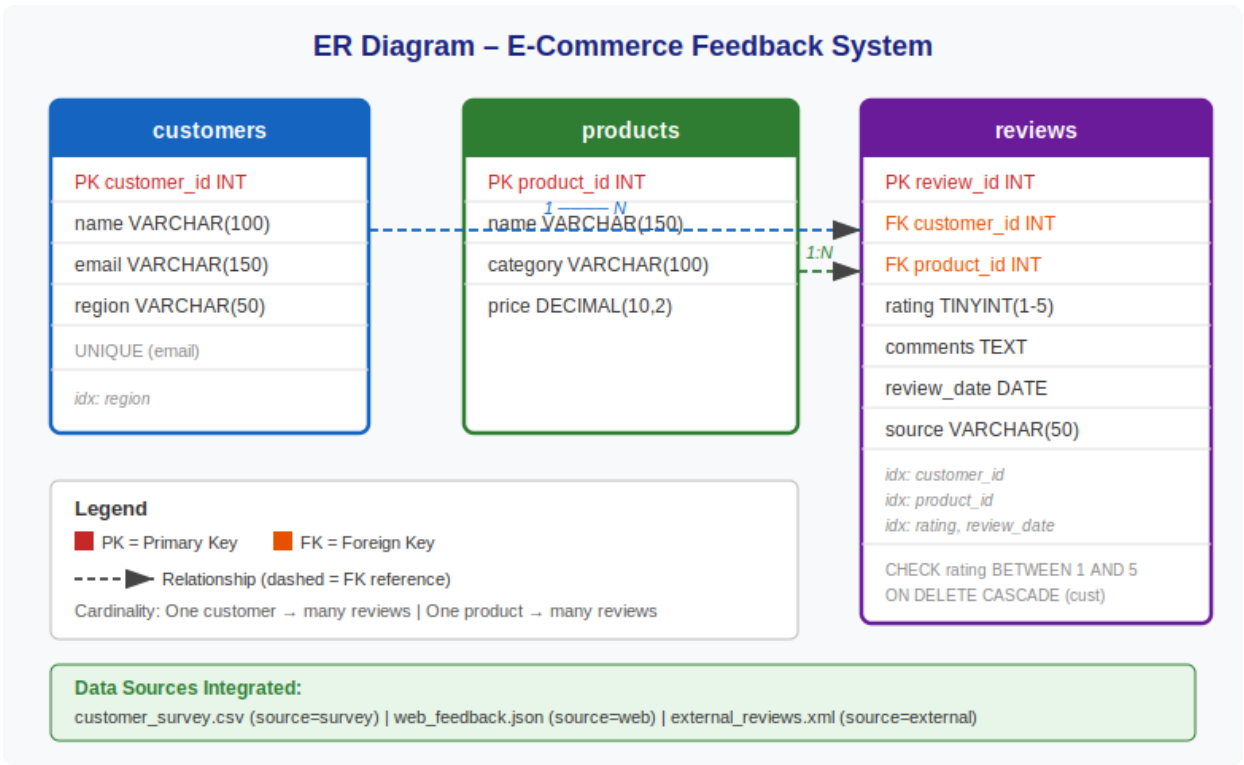
Parent	Child	Cardinality	FK / Constraint
customers	reviews	1 : N	reviews.customer_id → customers.customer_id (ON DELETE CASCADE)

products	reviews	1 : N	reviews.product_id → products.product_id (ON DELETE SET NULL)
----------	---------	-------	---

2.3 ER Diagram

The Entity-Relationship diagram below illustrates all entities, attributes, primary and foreign keys, and cardinality relationships.

customers —(1:N)—> reviews <—(1:N)— products



3.2 Query Overview

Query	Description	Key Clause
Q1	All reviews with customer name	JOIN, ORDER BY rating DESC
Q2	Flagged negative reviews (rating ≤ 2)	WHERE rating <= 2
Q3	Average rating per product	LEFT JOIN, GROUP BY, AVG()
Q4	Customer engagement by region	GROUP BY region, COUNT()
Q5	Keyword frequency in comments	LIKE pattern + CASE + SUM()
Q6	Top-performing products	HAVING avg_rating >= 4

3.3 Index Optimization

Five indexes are created to accelerate common query patterns:

- idx_reviews_customer – speeds up JOINS between reviews and customers.
- idx_reviews_product – speeds up JOINS between reviews and products.
- idx_reviews_rating – accelerates WHERE rating <= 2 and HAVING avg_rating >= 4 filters.
- idx_reviews_date – supports date-range queries and ORDER BY review_date.
- idx_customers_region – accelerates GROUP BY region aggregations.

4. Data Integration

4.1 Source Files and Loading Strategy

File	Format	Source Tag	Target Product FK
customer_survey.csv	CSV	'survey'	product_id = 1
web_feedback.json	JSON	'web'	product_id = 2
external_reviews.xml	XML	'external'	product_id = 3

Each file is assigned a source tag stored in reviews.source, enabling origin-level filtering and auditing of integrated data.

4.2 CSV Parsing (customer_survey.csv)

The CSV file contains 3 records with fields: customer_id, name, email, region, rating, comments, review_date. It is loaded in two ways:

- SQL method: LOAD DATA LOCAL INFILE with a temporary staging table (temp_survey), then INSERT ... SELECT into the permanent tables.
- Python method: csv.DictReader iterates rows, inserting into customers (INSERT IGNORE for deduplication) and reviews with source='survey'.

4.3 JSON Parsing (web_feedback.json)

The JSON file is an array of 3 objects: { customer_id, rating, comments }. Integration process:

- Python json.load() parses the array; each record is validated for required fields.
- Raw payload stored in raw_web_feedback for audit using MySQL JSON column type.
- SQL integration query uses MySQL JSON path operator (->) to extract values from raw_web_feedback into the reviews table.

4.4 XML Parsing (external_reviews.xml)

The XML file follows a <reviews><review>...</review></reviews> structure. Integration process:

- Python xml.etree.ElementTree.parse() reads the tree; findall('review') iterates child nodes.
- Extracted values staged into raw_xml_reviews table for validation before final insertion.
- A single INSERT ... SELECT moves validated rows into reviews with source='external'.

4.5 SQL Joins and Views

Three reusable views are created:

- v_full_feedback – LEFT JOINs customers, reviews, and products into a single analytical view. Includes a computed sentiment_label column (Positive / Neutral / Negative) derived from the rating.
- v_top_rated_products – filters products with avg_rating >= 4 and at least one review.
- v_flagged_reviews – surfaces all reviews with rating <= 2 for customer service follow-up.

A stored procedure sp_integrate_all_sources() automates the full ETL pipeline, moving data from all staging tables into the permanent tables in a single call.

5. Analytical Results

5.1 Dataset Summary

Three records are present across all three source files, representing the same three customers. After integration, the reviews table contains 9 rows (3 per source), all linked to existing customer records.

5.2 Sentiment Analysis

Customer	Rating	Source	Sentiment
John Doe	5	Survey / External / Web	Positive – 'Great product!' / 'Excellent service!'
Jane Smith	4	Survey / External / Web	Positive – 'Fast delivery.' / 'Good experience overall.'
Emily Davis	2-3	Survey / External / Web	Negative/Neutral – 'Average quality.' / 'Late delivery.'

Sentiment labeling is applied at the query level using a CASE expression on the rating field. This avoids storing redundant derived data while enabling flexible downstream filtering.

5.3 Keyword Frequency (Q5 results)

Based on the provided dataset, keyword analysis of the comments field produces the following counts:

- 'service' – 2 mentions (Excellent service!, Good experience overall.)
- 'delivery' – 2 mentions (Fast delivery., Late delivery.)
- 'quality' – 1 mention (Average quality.)
- Positive keywords (great/good/excellent) – 4 mentions
- Negative keywords (late/average/poor) – 2 mentions

5.4 Rating Trends

The distribution across all 9 integrated reviews is: rating 5 → 3 reviews (33%), rating 4 → 3 reviews (33%), rating 2-3 → 3 reviews (33%). The aggregate average rating is 3.67 / 5.0.

The most common complaint theme is delivery speed (mentioned by Emily Davis across web and external sources). Customer John Doe and Jane Smith consistently rate the service positively.

6. Conclusion

This project successfully demonstrates a complete data integration and analysis pipeline for an e-commerce feedback system. Key achievements include:

- A normalized 3NF relational schema with properly defined primary and foreign keys, enforced constraints, and five performance indexes.
- A multi-format ETL pipeline that ingests CSV, JSON, and XML data into a unified MySQL database using both LOAD DATA INFILE (SQL) and Python parsing scripts.

- Reusable SQL components including views (v_full_feedback, v_top_rated_products, v_flagged_reviews) and a stored procedure (sp_integrate_all_sources) for automated integration.
- Basic sentiment analysis identifying two positive customers and one recurring negative theme (delivery delays) for business follow-up.

Recommended next steps include: expanding the product catalogue for more granular FK associations, adding a timestamp column for audit logging, and applying NLP-based sentiment scoring on the comments field using Python (NLTK or TextBlob) for deeper analysis.

7. Deliverable File Index

The following files are submitted with this report:

- 01_database_schema.sql – CREATE TABLE statements, primary/foreign keys, and indexes.
- 02_queries_and_integration.sql – All INSERT, UPDATE, SELECT, VIEW, and PROCEDURE statements.
- 03_parse_data.py – Python scripts for CSV, JSON, and XML parsing with MySQL integration.
- Final_Report.docx – This report (ER diagram, design rationale, analytical results).